ENGINEERING BUILD REPORT · CONFIDENTIAL

The Unified Trading Intelligence Operating System

What was actually built. Three overlapping platforms — LETRA 37, F16 Raptor / Senseei, and Symphony — collapsed into one modular MQL5 operating system: a kernel, six engines, one shared market state, and a single deterministic per-bar pipeline. Plus three execution subsystems engineered specifically for this strategy: the Symphony phase authority, the PYRO thermal risk engine, and the TALON grip.

REPOSITORY	BRANCH	BUILD	TARGET
snxperfx1-dev / combo	falcon-os-platform	v3.27 · single-file	XAUUSD · M15

01 · EXECUTIVE SUMMARY

Three platforms. One operating system. Shipped.

FALCON OS is not a merge of three scripts. It is a complete redesign where every subsystem shares one market state and one event pipeline. Every calculation exists exactly once. The original tri-system duplication — physics, ATR, waves, liquidity, structure — is gone.

6	3	1×	0
Engines on one shared state & deterministic pipeline	Purpose-built execution subsystems (Symphony · PYRO · TALON)	Each calculation runs exactly once per candle	DRDWCT trimmer remnants — fully removed

WHAT WAS UNIFIED

The six-layer OS

Market · Memory · Intelligence · Decision · Execution · Visualization — each owning one domain, communicating through a kernel (shared state + event bus + scheduler + config + logging + persistence). LETRA's physics, F16's Invisible Network & Senseei, and Symphony's execution all live in one codebase, computed once.

WHAT WAS ENGINEERED NEW

The execution trinity

Symphony became the single phase/direction authority for precision Phase 3/4 entries. PYRO models each stacked campaign as a body with heat. TALON replaced basic trailing with a curve-convergent structural grip. Together they handle a stacking, curve-driven strategy that generic risk tools cannot.

THE GOVERNING LAW

Phases are outputs, never inputs. Direction emerges from ownership — it is never voted. Risk is evaluated per campaign and never netted across opposite directions. Every subsystem owns exactly one question; nothing calculates twice.

This dossier documents the implemented system as of build v3.27. Source is a modular tree under `FALCON_OS/` plus a generated single-file build, `FalconOS_AllInOne.mq5`, intended for compilation in MetaEditor 5.

02 · SYSTEM ARCHITECTURE

Six layers. One direction of flow.

Data flows down through the layers; nothing crosses a boundary except through the shared `g_state` object. Below, each layer as implemented, with its real subsystems.

01 Market Engine — OBSERVES

MarketEngine.mqh · source: LETRA 37

Physics · Structure · Liquidity · Inducement · Convexity / ARC · Wave · FU Candle · Order Blocks · Supply / Demand · HTF Stack

02 Memory Engine — REMEMBERS

MemoryEngine.mqh · source: F16 Raptor

Invisible Network · Node Registry · Conversation Web · Authority · Curve Tree · Wave Matrix · FEZ · FRZ · Campaign Ownership · Participants

03 Intelligence Engine — REASONS

IntelligenceEngine.mqh · source: LETRA + F16

Energy Resolution · Belief · Forecast (RFE/FGE) · Hypothesis · Prediction · Validation · Liquidation Wave · Entry Cycle · Threat / Opportunity · Story

04 Decision Engine — DECIDES

DecisionEngine.mqh · source: Senseei / Chief Strategist

Senseei · Chief Strategist · Campaign AI · Master Chief · BUY · SELL · WAIT · ATTACK · DEFEND · EXIT · SCALE · NO TRADE

05 Execution Engine — EXECUTES ONLY

ExecutionEngine.mqh + SymphonyEngine.mqh + ThermalRiskEngine.mqh · source: Symphony

Symphony Phase Entries · PYRO Thermal Risk · TALON Grip · Lot Engine · ARC / Institutional Exit · Session Filter · Drawdown Protection · Partial Close

06 Visualization Engine — DISPLAYS

Visualization.mqh · replaces all prior dashboards

Overview · Physics · Structure · Network · Curve · Campaign · Wave · HTF · Risk (PYRO/TALON) · Execution · Performance · Diagnostics

03 · THE KERNEL

The infrastructure beneath everything.

A lightweight kernel ties every module together — not by coupling them, but by giving them shared services they never rebuild. Implemented across the `Kernel/` directory.

Shared State Manager

`FalconState.mqh` — the single `g_state` contract. No module owns state belonging to another. One ATR, one wave, one campaign owner, system-wide.

Event Bus

`FalconEventBus.mqh` — publish/subscribe. Modules react to events (bar, spawn, phase change, risk breach) rather than polling. New engines plug in without touching the pipeline.

Scheduler

`FalconOS.mq5` pipeline — the same deterministic sequence every bar. Same inputs → same outputs. No race conditions.

Configuration Service

`FalconConfig.mqh` — every tunable declared exactly once in `g_cfg`, with LIVE / BACKTEST / RESEARCH profiles.

Logging & Diagnostics

`FalconLog.mqh` — structured logs, per-module timing, health checks surfaced in the Diagnostics tab.

Persistence Layer

`FalconPersistence.mqh` — equity/drawdown tracking, network & campaign memory, performance metrics across sessions.

SERIES & MATH — SINGLE IMPLEMENTATION

`FalconSeries.mqh` holds the one ATR (`FalconATR`), one pivot test (`FalconIsPivotHigh/Low`), and shared price series (`gOpen/gHigh/gLow/gClose`). Every engine reads these — there is no second ATR anywhere in the system.

04 · MASTER PIPELINE

Every bar.
Same sequence. Always.

One fixed pipeline runs on every confirmed candle. The Symphony phase bridge, PYRO, and TALON are woven into the deterministic order so nothing recalculates and nothing fires out of sequence.

TRIGGER

New Candle

Bar context written to g_state; pipeline locks for this cycle.

MARKET LAYER

Market Engine — physics → structure → liquidity → convexity → wave → FU → OB → S/D → HTF

All raw market geometry observed and written once.

PHASE AUTHORITY

Symphony Update + Bridge

Impulse + Phase 1–4 + ARC computed, then bridged onto canonical g_state.wave (phase/direction/flip/origin/objective). The single phase truth.

MEMORY LAYER

Memory Engine — Network → Curve → Wave Matrix → FEZ → FRZ → Campaign → Participants

Ownership flips with Symphony's phase; nothing votes.

INTELLIGENCE LAYER

Energy → Belief → Forecast → Hypothesis → Prediction → Validation → Entry Cycle → Story

Continuous probabilities, not binary states.

DECISION LAYER

Senseei → Chief Strategist → Campaign AI → Master Chief

One unambiguous verdict per bar.

EXECUTION LAYER · A

Execution Engine — exposure snapshot + drawdown kill-switch

Translates directives to orders; never decides.

EXECUTION LAYER · B

PYRO Thermal Risk — heat · admissions · thermostat · catastrophe flatten

Per-campaign basket risk computed before any new stack.

EXECUTION LAYER · C

Symphony Manage — TALON grip → ARC partial → composite exit → Phase 3/4 entries

Entries gated by PYRO admission; stops owned by TALON.

PERSISTENCE + OUTPUT

Persistence tick → Visualization Engine

12 tabs read finalized state. Zero recalculation.

05 · MASTER SHARED STATE

One object. Every module reads from it.

The `g_state` object is the single source of truth. The implemented domains, with the new risk sub-state added for PYRO & TALON.

Physics

ATR · Velocity · Accel
Convexity (+ smoothed)
Efficiency · Displacement
Energy · Compression · Expansion

Structure

Trend · HH/HL/LH/LL
Swing High / Low
BOS · CHoCH
Break strength

Liquidity

Pools · Sweeps
Sweep probability · pressure
Inducement · false CHoCH
Acceptance · score

Wave

Phase · prevPhase · direction
Completion · energy · age
Flip top/bot · origin · objective
Symphony mode + phases

HTF / FU

Per-TF direction · alignment
Stack dir · owner TF
FU candle · zone · confidence
Fractal agreement

Network

Nodes · authority
Conversation graph
Pressure · bias · next node
Dormancy · revisits

Curve / Campaign

Curve tree · owner dir/TF
Campaign owner · institution
Control · remaining energy
Participants

Intelligence

Belief · hypothesis
Prediction · validation
Exec probability · threat
Resolution · entry cycle

Execution + Risk

Entry · stop · target · lots
Trade / exit state
PYRO campaign heat ×2
TALON grip + stage ×2

SINGLE SOURCE OF TRUTH — ENFORCED

No scenario exists where two modules hold a different ATR, wave state, or campaign direction. The Symphony bridge writes the canonical phase; the Market Engine supplies geometry descriptors; everyone reads the same object.

06 · PRECISION ENTRIES

Symphony — the single phase & direction authority.

Symphony's proven curvature/retracement phase engine was ported verbatim onto FALCON's shared series, then made the primary order logic — and the single phase truth the entire OS reasons on.

THE ENGINE

Impulse + Phases 1–4

A confirmed impulse anchors a campaign (high→low for shorts, low→high for longs). Retracement fraction then classifies Phase 1 (impulse) → 2 (retrace) → 3 (return into zone) → 4 (breakout). Entries fire on Phase 3 & Phase 4 only, with Symphony's exact stop placement: anchor ± ATR × 0.25.

THE BRIDGE

One phase truth, system-wide

The bridge maps Symphony's mode + Phase 1–4 onto canonical `g_state.wave` (phase, direction, flip zone, origin, objective, completion, dominance). Memory ownership, Intelligence, Decision, Execution and Visualization all read this — no second phase engine survives downstream.

```
// Phase map (direction-aware) → canonical PH_*
 phase 1  impulse        → PH_EXPANSION
 phase 2  retracing      → PH_RETRACEMENT
 phase 3  return-to-zone → PH_DEMAND_RETURN  (long) / PH_SUPPLY_RETURN (short)
 phase 4  breakout       → PH_NEW_HIGH       (long) / PH_NEW_LOW       (short)
 // dominanceTransfer keyed to phase ≥ 3 → campaign ownership flips with Symphony
```

WHY THIS MATTERS

Before the bridge, two phase engines ran in parallel (Market Engine + Symphony) producing competing direction. Now the Market Engine observes geometry; Symphony owns the phase. Direction emerges from the flip-driven campaign owner — never a vote.

Lot sizing uses Symphony's contract-value model — `riskCash / (dist × contractValue)` — capped by `InpMaxLots`. The legacy tick-value path that mis-sized orders was removed.

07 · THE RISK ENGINE

PYRO — Campaign Thermodynamics.

A risk model built for how this algo actually trades: precision entries that stack into a directional campaign. Each campaign is treated as a physical body that carries heat.

```
heat = adverseExcursion(blended basket, ATR) × fragility(stackCount, totalLots)
```

A winning basket runs near-zero heat no matter how large — it is house money, never trimmed. An underwater, heavily-stacked basket overheats fast. Heat is the single scalar that governs admission of every new stack:

OPEN	THROTTLED	FROZEN	DE-RISK
cool — full stacks	warming — shrink size	hot / maxed — no stacks	critical — flatten

NOVEL FOR STACKING

Anti-martingale governor

What kills stacking systems is averaging down into a reversal. PYRO forbids adding beyond `MaxAvgDownStacks` while underwater, and shrinks each admitted stack inversely to heat.

NEVER NETS

Dual-campaign thermostat

Long-heat and short-heat tracked separately. If both overheat at once (whipsaw trap), all admissions freeze. Account heat derives from equity drawdown vs peak.

THE CONTRAST WITH THE OLD DRDWCT ENGINE

DRDWCT trimmed everything — including winners — via VaR/UDS. PYRO never trims a winner: heat is ~0 in profit, so the only forced close is a true runaway (deeply underwater + large) at critical heat, exactly when a stacking book must be cut. The admission gate `TR_AdmitLots()` sizes every Symphony entry.

08 · BREAKEVEN & TRAILING

TALON — the curve-convergent structural grip.

Basic ATR trailing is the wrong tool for a curve-driven, stacking strategy — it chokes the trade on a healthy retrace or gives back half the move at the turn. TALON drives the stop with the same intelligence that drives entries.

FORMING	BREAKEVEN	RIDING	CONVERGING	TERMINAL
on structural stop	earned by structure	behind swing pivots	tightening to target	tightest — profit rolls over

- Earned breakeven — locks only when structure confirms (a BOS in the campaign direction) or the fleet reaches `TalonBeATR` in favor. No more getting tagged on a phase-2 retrace the engine considers healthy.
- Structural ratchet — the stop rides behind confirmed swing pivots (higher-lows for longs / lower-highs for shorts), never a fixed distance behind raw price.
- Curve convergence — the trail distance contracts as price nears the ARC target / wave objective and as geometry capacity drains. Far = wide (let it run); near = tight (bank before reversal).
- Phase / thermal coupling — hard-tightens at terminal phase (NEW_HIGH / NEW_LOW) or when the campaign's profit velocity rolls over.
- Basket-level — one grip for the whole fleet, off blended cost. Resolves the per-leg vs basket duplication entirely.

CLEAN RESPONSIBILITY SPLIT — NO DUPLICATE DECISIONS

Symphony owns entries · ARC partial · composite reversal exit. **TALON** owns all protective stops (breakeven + trailing). **PYRO** owns admission/sizing · thermostat · catastrophe flatten. The duplicate basket-breakeven that previously lived in PYRO was removed — one owner per decision.

09 · AUTHORITY & REMOVAL

One owner per decision. Nothing duplicated.

Every decision in the execution layer has exactly one authority. The table below is the contract.

DECISION	AUTHORITY	MUST NEVER
Phase & direction	Symphony (bridged to g_state.wave)	be recomputed by Market Engine
Entry timing	Symphony Phase 3 / 4	fire without PYRO admission
Stack size / admission	PYRO TR_AdmitLots	average down past the limit
Protective stop (BE + trail)	TALON grip	be touched by PYRO or Symphony
Profit banking	Symphony ARC partial	conflict with the TALON ratchet
Reversal exit	Symphony composite (ARC + inst + phase)	close a healthy campaign
Catastrophe close	PYRO (critical heat) + drawdown kill-switch	trim a winning basket

REMOVED IN FULL

The DRDWCT risk engine

The original Symphony DRDWCT VaR/UDS/gamma trimmer closed open winners and was removed completely — first the logic, then all dead scaffolding:

- VaR scenario engine
- UDS metrics
- gamma / RD / SAG
- micro-bomb trim loop
- EE_Metrics / EE_VarResult structs
- logistic bottom-layer state
- var2/var3/udsMax/anyBomb fields
- InpEnableRiskEng · InpRdLimit

VERIFICATION

A full source sweep for the removed symbols returns zero live references. The only remaining mentions are comments noting the removal. Risk is now owned entirely by PYRO + TALON + the drawdown kill-switch.

10 · BUILD, FILES & OPERATION

How it ships. How it runs.

SOURCE TREE

Modular — FALCON_OS/

```
Kernel/
   FalconConfig · FalconState
   FalconSeries · FalconEventBus
   FalconLog · FalconPersistence
Engines/
   MarketEngine · MemoryEngine
   IntelligenceEngine · DecisionEngine
   ExecutionEngine
   ThermalRiskEngine (PYRO)
   SymphonyEngine (phase + TALON)
   Visualization
FalconOS.mq5   // orchestrator
```

DELIVERY

Single-file build

`FalconOS_AllInOne.mq5` — every kernel + engine concatenated in dependency order, includes/properties stripped, version-stamped. Drop into MetaEditor 5 and compile (the Linux build sandbox cannot compile MQL5, so balance is verified statically each build).

RUN PROFILE

XAUUSD · M15. For full backtests set `InpSessionFilter = false`. `InpUseSymphony`, `InpUseThermalRisk`, `InpUseTalon` default on.

VERSION HISTORY

BUILD	MILESTONE
v3.2x	FALCON OS unified — 6 engines, kernel, shared state, deterministic pipeline
v3.22	Symphony Phase 3/4 engine ported as the precision entry/exit authority
v3.23	Symphony made the single phase/direction source of truth (bridge)
v3.24	Symphony trade management added (breakeven · trailing · ARC partial)
v3.25	PYRO Campaign Thermodynamics risk engine for stacked entries
v3.26	DRDWCT scaffolding scrubbed completely
v3.27	TALON curve-convergent structural grip; duplicate breakeven removed

Repository snxperfx1-dev/combo · branch falcon-os-platform. Each build is committed and pushed; the single-file artifact is pinned per commit for review.

BUILT FOR WHAT COMES NEXT

A system designed to receive the future without disruption.

The kernel is the foundation. The shared state is the contract. The pipeline is the discipline. Symphony owns the phase, PYRO owns the heat, TALON owns the grip — each one job, one truth, one way to communicate. New models, data feeds, markets and instruments plug into a core designed to receive them.

FALCON OS is a complete redesign from three overlapping codebases into a coherent, modular trading operating system — with a shared architecture, a unified state model, a deterministic pipeline, and risk & trade-management subsystems engineered specifically for precision stacked-campaign execution.